

DBMS Unit 5 - Database Administration

UNIT - V: DATABASE ADMINISTRATION

MSBTE K-Scheme | Diploma CO | SY - Database Management System

5.1 INTRODUCTION TO DATABASE ADMINISTRATION

Database Administration refers to the set of activities performed to manage, maintain, secure, and optimize a database system. The person responsible for these tasks is called the **Database Administrator (DBA)**.

Responsibilities of a DBA: - Installing and upgrading the DBMS software - Creating and managing user accounts and privileges - Ensuring database security - Monitoring performance and tuning the database - Taking regular backups and planning recovery - Managing storage and space allocation - Maintaining data integrity and consistency

Types of Database Users

Type of User	Description
Database Administrator (DBA)	Manages the entire database system — security, backup, performance, user accounts
Application Programmers	Write application programs using host languages combined with DML to interact with the database
Sophisticated Users	Interact with the system directly using a query language (e.g., analysts writing SQL queries)
Naive/End Users	Interact with the database through permanent application programs/forms (e.g., a bank clerk using a banking form)
Specialized Users	Write specialized database applications, such as for CAD systems, knowledge-based systems
Casual Users	Occasionally access the database, using different queries each time

Create and Delete Users

Create User:

```
CREATE USER username IDENTIFIED BY password;
```

Example:

```
CREATE USER student_user IDENTIFIED BY pass123;
```

Delete (Drop) User:

```
DROP USER username;
```

To drop a user along with all the objects owned by them:

```
DROP USER username CASCADE;
```

Alter User (change password):

```
ALTER USER username IDENTIFIED BY new_password;
```

Assign Privileges to Users

Privileges are permissions given to a user to perform specific actions on database objects. There are two main categories:

Privilege Type	Description	Examples
System Privileges	Allow a user to perform actions on the database as a whole	CREATE SESSION, CREATE TABLE, CREATE USER
Object Privileges	Allow a user to perform actions on a specific database object	SELECT, INSERT, UPDATE, DELETE on a table

Granting Privileges:

```
GRANT CREATE SESSION, CREATE TABLE TO student_user;  
GRANT SELECT, INSERT ON Student TO student_user;
```

Revoking Privileges:

```
REVOKE INSERT ON Student FROM student_user;
```

Granting with Admin Option (allows the user to further grant the privilege to others):

```
GRANT SELECT ON Student TO student_user WITH GRANT OPTION;
```

5.2 TRANSACTION

Concept of Transaction

A **transaction** is a **logical unit of work** that comprises one or more SQL operations (INSERT, UPDATE, DELETE, SELECT), which must be executed **as a whole** — either all operations complete successfully, or none of them take effect.

Example: A bank fund transfer from Account A to Account B involves two operations: 1. Debit amount from Account A 2. Credit amount to Account B

Both operations must succeed together; if either fails, the entire transaction must be undone to maintain consistency.

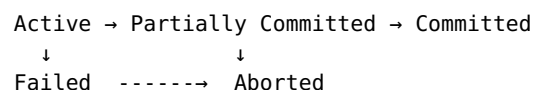
Properties of a Transaction (ACID Properties)

Property	Description
Atomicity	The transaction is treated as a single, indivisible unit — either all operations are executed, or none are (all-or-nothing)
Consistency	The database must move from one valid (consistent) state to another valid state after the transaction
Isolation	Concurrent transactions must execute independently without interfering with one another; intermediate results of one transaction are not visible to others
Durability	Once a transaction is committed, its changes are permanent , even in the event of a system failure

(Remember using the acronym: **ACID**)

States of a Transaction

A transaction passes through several states during its execution lifecycle:



State	Description
Active	The initial state; the transaction is currently executing
Partially Committed	The transaction has executed its final statement, but changes are not yet permanently saved
Committed	The transaction has completed successfully; all changes are permanently saved to the database
Failed	The transaction cannot proceed further due to an error/hardware failure
Aborted	The transaction has been rolled back, and the database is restored to its state before the transaction began

5.3 DATABASE BACKUP

Types of Failures

Failure Type	Description
Transaction Failure	Occurs due to logical errors (e.g., invalid input, overflow) or system errors (e.g., deadlock) within a transaction
	Occurs due to hardware, software, or operating system failure, causing loss of data in

System Crash	main memory (but disk/secondary storage remains intact)
Disk Failure	Occurs due to physical damage to the disk (head crash) or corruption, causing loss of stored data

Causes of Failure

1. **Hardware failure** - power outage, disk crash, memory failure
2. **Software failure** - bugs in DBMS or operating system
3. **Human errors** - accidental deletion, incorrect operations by users
4. **Natural disasters** - fire, flood, earthquake affecting physical infrastructure
5. **Network failure** - in distributed database systems
6. **Malicious operations** - unauthorized access, viruses, sabotage

Introduction to Database Backup

Backup is the process of creating a copy of the database that can be used to **restore** the original data in case of loss, corruption, or failure. Regular backups are essential to ensure **data availability and disaster recovery**.

Why backup is important: - Protects against data loss due to hardware/software failure - Enables recovery from accidental or malicious data deletion - Ensures business continuity

Types of Database Backup

1. Physical Backup

A physical backup involves copying the **actual physical files** of the database — such as data files, control files, and redo log files — at the operating system level.

Sub-types: - **Full Backup** - copies the entire database - **Incremental Backup** - copies only the data that has changed since the last backup - **Differential Backup** - copies data changed since the last full backup

Advantages: Fast to restore since it is a direct file-level copy; suitable for large databases.

2. Logical Backup

A logical backup involves extracting **logical data** (such as table structures, data, and objects) using SQL statements or utility tools (e.g., Oracle's EXPDP/EXPORT), rather than copying raw files.

Advantages: Portable across different platforms/versions; allows selective backup of specific tables/schemas.

Comparison:

Basis	Physical Backup	Logical Backup
What is copied	Actual data files at OS level	Logical objects (tables, schemas) via SQL/export tools
Speed	Faster	Slower (relatively)
Restore Speed	Faster	Slower
Portability	Less portable (platform dependent)	More portable

Granularity	Whole database/files	Selective (specific tables/schema)
-------------	----------------------	------------------------------------

5.4 DATA RECOVERY

Recovery Concepts

Recovery is the process of restoring the database to a **consistent and correct state** after a failure has occurred, using backups and log files.

- The DBMS maintains a **transaction log (log file)** that records all changes made to the database (before and after values), which is used during recovery.
- The main goal of recovery is to ensure the **Atomicity** and **Durability** properties of transactions are preserved even after failure.

Recovery Techniques

1. Roll Forward (Redo)

- **Roll forward** (also called REDO) is used to **reapply** the changes of **committed transactions** that were made in memory but not yet written to the physical database files at the time of a crash.
- Uses the transaction log to redo/reapply completed operations, bringing the database up to date.
- Ensures that no committed work is lost.

2. Rollback (Undo)

- **Rollback** (also called UNDO) is used to **reverse/undo** the changes made by **incomplete (uncommitted) transactions** at the time of failure.
- Ensures that partial changes from a failed or aborted transaction do not corrupt the database, maintaining consistency.
- Can also be manually invoked using the ROLLBACK command to cancel a transaction.

Recovery Process (typical sequence):

1. Restore the last full backup
2. Roll Forward – redo all committed transactions from the log
3. Rollback – undo any incomplete/uncommitted transactions
4. Database reaches a consistent state

Roll Forward vs Rollback:

Basis	Roll Forward (Redo)	Rollback (Undo)
Applied to	Committed transactions	Uncommitted/incomplete transactions
Purpose	Reapply lost changes	Reverse partial changes
Direction	Moves database forward using logs	Moves database backward to prior consistent state

5.5 OVERVIEW OF ADVANCED DATABASE CONCEPTS

1. Data Warehouse

A **Data Warehouse** is a large, centralized repository that stores **integrated data** collected from multiple heterogeneous sources, specifically designed to support **decision-making, reporting, and analysis (OLAP)** rather than daily transaction processing (OLTP).

Characteristics: Subject-oriented, Integrated, Time-variant, Non-volatile (data is not updated/deleted once loaded).

Example use: A retail company consolidating sales data from all its stores to analyze yearly trends.

2. Data Lakes

A **Data Lake** is a centralized storage repository that can store **vast amounts of raw data** in its **native/original format** — structured, semi-structured, or unstructured — until it is needed for analysis.

- Unlike a data warehouse (which stores processed/structured data), a data lake stores **raw** data.
- Follows a “**schema-on-read**” approach (structure is applied only when data is read/used), unlike a warehouse which uses “schema-on-write.”

Example use: Storing raw log files, images, videos, and sensor data for future big-data analytics.

3. Data Mining

Data Mining is the process of discovering **hidden patterns, correlations, trends, and useful knowledge** from large volumes of data using statistical, mathematical, and machine learning techniques.

Common data mining tasks: - Classification, Clustering, Association Rule Mining, Regression, Anomaly Detection

Example use: An e-commerce company analyzing purchase history to recommend products (“customers who bought this also bought...”).

4. Big Data

Big Data refers to extremely large and complex datasets that cannot be processed or managed using traditional database tools, characterized by the **5 V's**:

V	Meaning
Volume	Massive amount of data (terabytes/petabytes)
Velocity	High speed at which data is generated and processed
Variety	Different formats of data — structured, semi-structured, unstructured
Veracity	Uncertainty/quality/trustworthiness of data
Value	Usefulness of the extracted information

Common Big Data tools: Hadoop, Apache Spark, Hive.

5. MongoDB

MongoDB is a popular **NoSQL, document-oriented database** that stores data in flexible, JSON-like documents called **BSON (Binary JSON)**, instead of traditional rows and columns.

Key features: - **Schema-less** – documents in the same collection can have different structures - Stores data in **collections** (equivalent to tables) containing **documents** (equivalent to rows) - Highly **scalable** (supports horizontal scaling/sharding) - Suitable for applications with rapidly changing or unstructured data

Example document:

```
{
  "Roll_No": 1,
  "Name": "Aditi",
  "Skills": ["SQL", "Python"]
}
```

6. DynamoDB

DynamoDB is a fully managed **NoSQL key-value and document database** provided by **Amazon Web Services (AWS)**, designed for high performance and seamless scalability.

Key features: - Stores data as **key-value pairs**, where each item is uniquely identified by a **primary key** - Fully managed – AWS handles server provisioning, patching, and scaling automatically - Offers **high availability** and **low-latency** performance at any scale - Suitable for web, mobile, gaming, and IoT applications requiring consistent, fast performance

Comparison - MongoDB vs DynamoDB:

Basis	MongoDB	DynamoDB
Type	Document-oriented NoSQL database	Key-value / document NoSQL database
Hosting	Self-hosted or MongoDB Atlas (cloud)	Fully managed by AWS only
Data format	BSON (JSON-like documents)	Key-value pairs / JSON documents
Scaling	Manual/auto sharding	Fully automatic scaling

Quick Revision Points

- **DBA** manages the database; **user types:** DBA, Application Programmers, Sophisticated, Naive/End, Casual
- **Privileges:** System (CREATE SESSION, CREATE TABLE) vs Object (SELECT, INSERT on specific table)
- **Transaction = logical unit of work;** must satisfy **ACID:** Atomicity, Consistency, Isolation, Durability
- **Transaction states:** Active → Partially Committed → Committed (or Failed → Aborted)
- **Failures:** Transaction failure, System crash, Disk failure
- **Backup types:** Physical (file-level copy) vs Logical (SQL/export-based, portable)
- **Recovery:** Roll Forward/Redo (reapply committed changes) vs Rollback/Undo (reverse uncommitted changes)
- **Data Warehouse** = structured, processed data for analysis; **Data Lake** = raw data, schema-on-read
- **Data Mining** = discovering patterns/knowledge from data
- **Big Data** = 5 V's — Volume, Velocity, Variety, Veracity, Value

- **MongoDB** = document-based NoSQL (BSON); **DynamoDB** = AWS-managed key-value/document NoSQL